Pannon Egyetem

Mesterséges Intelligencia megoldásokat fejlesztő adat- és rendszertudományi szakember képzés

Szakdolgozat beszámoló

Kriskó Nándor

Témavezető:
Dr. Czúni László

Pannon Egyetem
University of Pannonia

Sávdetektálás mélytanulásos módszerek alkalmazásával

- Feladat bemutatása
- Elérhető módszerek
- State of the art modellek
- Alkalmazott U-Net modell bemutatása
- Optimalizálás, finomhangolás
- Bővítés gépjármű detekcióval

Pannon Egyetem
University of Pannonia

A feladat

- Sávdetektálás módszertanainak vizsgálata
- Adatbázisok keresése
- Sávdetektáló módszer implementálása
- A modell optimalizálása
- Közlekedő autók felismerése -> objektum detektálás
- A két rendszer összehangolása és futtatása

Pannon Egyetem
University of Pannonia

A sávdetektálás nehézségei

- **Változó időjárási körülmények**
- **Változó közlekedése körülmények**
- **Útburkolati jelölések hibái**

State of the art és benchmark

Dataset függő, hogy melyik az aktuális legjobb

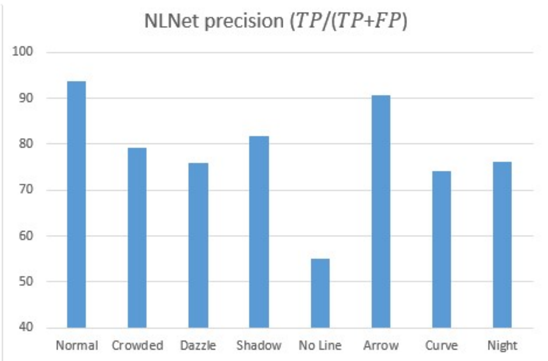
Dataset	Model	Method
TUSimple	RESA	Segmentation
CULane	CLRNet	"Anchor box"
CurveLanes	CondLaneNet	"Anchor box"
Llamas	CLRNet	"Anchor box"
BDD100k	YoloPv2	Segmentation

A környezeti körülmények erősen befolyásolják az eredményeket.

State of the art és benchmark

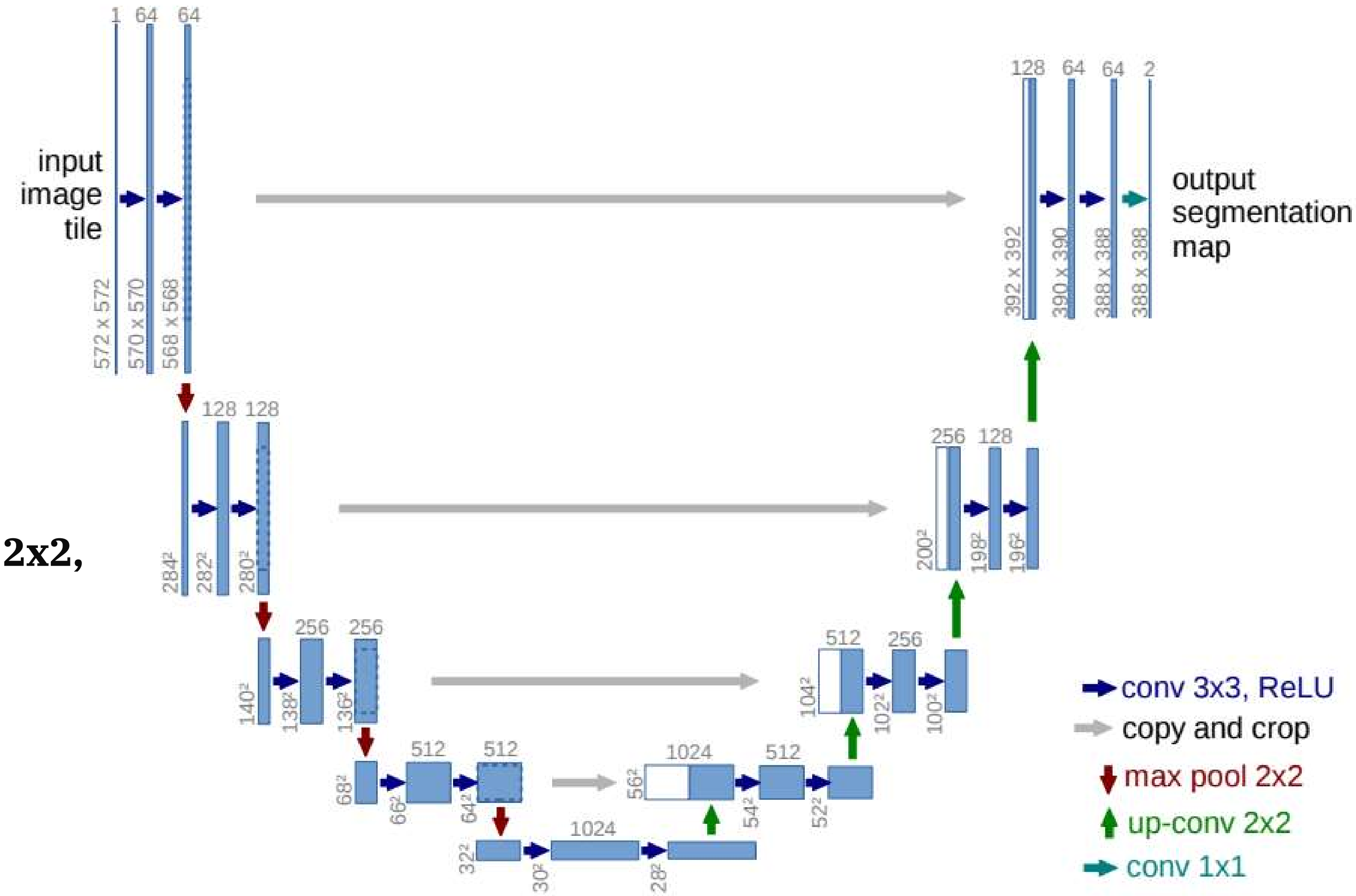
CULane Dataset

Method	Backbone	mF1	F1@50	F1@75	FPS	Normal	Crowded	Dazzle	Shadow	No Line	Arrow	Curve	Cross	Night
SCNN	VGG16	38.84	71.60	39.84	7.5	90.60	69.70	58.50	66.90	43.40	84.10	64.40	1990	66.10
RESA	ResNet34	-	74.50	-	45.5	91.90	72.40	66.50	72.00	46.30	88.10	68.60	1896	69.80
RESA	ResNet50	47.86	75.30	53.39	35.7	92.10	73.10	69.20	72.80	47.70	88.30	70.30	1503	69.90
FastDraw	ResNet50	-	-	-	90.3	85.90	63.60	57.00	69.90	40.60	79.40	65.20	7013	57.80
E2E	ERFNet	-	74.00		-	91.00	73.10	64.50	74.10	46.60	85.80	71.90	2022	67.90
UFLD	ResNet18	38.94	68.40	40.01	282	87.70	66.00	58.40	62.80	40.20	81.00	57.90	1743	62.10
UFLD	ResNet34		72.30	-	170	90.70	70.20	59.50	69.30	44.40	85.70	69.50	2037	66.70
PINet	Hourglass	46.81	74.40	51.33	25	90.30	72.30	66.30	68.40	49.80	83.70	65.20	1427	67.70
LaneATT	ResNet18	47.35	75.13	51.29	153	91.17	72.71	65.82	68.03	49.13	87.82	63.75	**1020**	68.58
LaneATT	ResNet34	49.57	76.68	54.34	129	92.14	75.03	66.47	78.15	49.39	88.38	67.72	1330	70.72
LaneATT	ResNet122	51.48	77.02	57.5	20	91.74	76.16	69.47	76.31	50.46	86.29	64.05	1264	70.81
LaneAF	ERFNet	48.6	75.63	54.53	24	91.10	73.32	69.71	75.81	50.62	86.86	65.02	1844	70.90
LaneAF	DLA34	50.42	77.41	56.79	20	91.80	75.61	71.78	79.12	51.38	86.88	72.70	1360	73.03
SGNet	ResNet18	-	76.12	-	117	91.42	74.05	66.89	72.17	50.16	87.13	67.02	1164	70.67
SGNet	ResNet34	-	77.27	-	92	92.07	75.41	67.75	74.31	50.90	87.97	69.65	1373	72.69
FOLOLane	ERFNet	-	78.80	-	40	92.70	77.80	75.20	79.30	52.10	89.00	69.40	1569	74.50
CondLane	ResNet18	51.84	78.14	57.42	173	92.87	75.79	70.72	80.01	52.39	89.37	72.40	1364	73.23
CondLane	ResNet34	53.11	78.74	59.39	128	93.38	77.14	71.17	79.93	51.85	89.89	73.88	1387	73.92
CondLane	ResNet101	54.83	79.48	61.23	47	93.47	77.44	70.93	80.91	54.13	90.16	**75.21**	1201	74.80
CLRNet	DLA34	55.64	80.47	62.78	94	93.73	**79.59**	75.30	**82.51**	54.58	90.62	74.13	1155	75.37
NLNet(ours)	DLA34	**55.75**	**80.49**	**63.16**	83	**93.74**	79.19	**75.85**	81.83	**55.03**	**90.69**	74.14	1291	**76.17**



Kísérlet szegmentáló hálózatokkal: U-Net

- **2015-ben alkotta meg O. Ronneberger et al.**
- **Encoder – decoder felépítés**
- **Encoder: Conv2D 3x3, Maxpooling2D 2x2**
- **Decoder: Conv2DTranspose 2x2, concatenate , Conv2D 3x3**
- **Skip connections**



Pannon Egyetem
University of Pannonia

TUSimple adatbázis

- 6408 db autópályán készült kép
- 3626 db train + 2782 db teszt
- 1280x720 felbontás
- 1 sec videó -> 20 db kép -> utolsó van annotálva
- annotálás: .json file ; pixelek, melyeken sáv található

```
{
  "lanes": [
        [-2, -2, -2, -2, 632, 625, 617, 609, 601, 594,
        [-2, -2, -2, -2, 719, 734, 748, 762, 777, 791,
        [-2, -2, -2, -2, -2, 532, 503, 474, 445, 416,
        [-2, -2, -2, 781, 822, 862, 903, 944, 984, 102
       ],
  "h_samples": [240, 250, 260, 270, 280, 290, 300, 310,
  "raw_file": "path_to_clip"
}
```

Pannon Egyetem
University of Pannonia

U-net alkalmazása

- **Adatok: TUSimple dataset**
- **Input kép: 1280x720x3 -> 256x256x3**
- **Input maszk: .json -> 256x256x1 (bináris)**
- **Train: 1800 db kép**
- **Teszt: 320 db kép**

```
Total params: 1,941,105 (7.40 MB)
Trainable params: 1,941,105 (7.40 MB)
Non-trainable params: 0 (0.00 B)
```

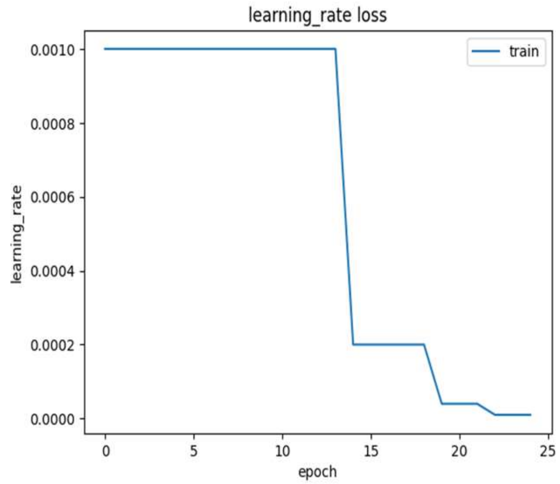
```python
with strategy.scope():
    model = create_unet_model()
    model.compile(optimizer='adamw', loss='dice', metrics=['accuracy'])

callbacks = [
        tf.keras.callbacks.CSVLogger('/kaggle/working/log_tuned_adamw_dice_batch16.csv', append=False),
        tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=2, verbose=1, min_delta=0.001, min_lr=0.00001),
        tf.keras.callbacks.TensorBoard(log_dir='/kaggle/working/logs'),
        tf.keras.callbacks.EarlyStopping(patience=5, monitor='val_loss'),
        tf.keras.callbacks.ModelCheckpoint('/kaggle/working/model_for_lane_tuned_adamw_dice_batch16.keras', verbose=1, save_best_only=True)]

results = model.fit(X_train, Y_train, batch_size=16, epochs=25,steps_per_epoch=62,validation_split=0.1, callbacks=callbacks)
```

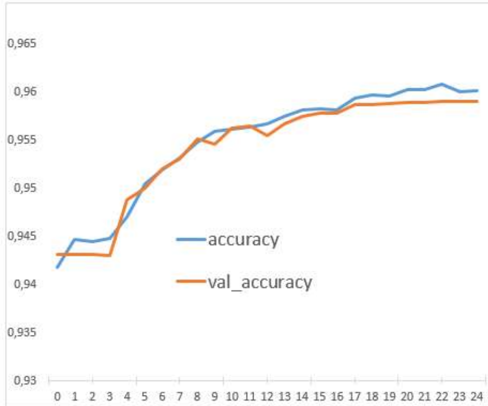
```python
s = inputs

# Contraction path
c1 = tf.keras.layers.Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(s)
c1 = tf.keras.layers.Dropout(0.1)(c1)
c1 = tf.keras.layers.Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c1)
p1 = tf.keras.layers.MaxPooling2D((2, 2))(c1)

c2 = tf.keras.layers.Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p1)
c2 = tf.keras.layers.Dropout(0.1)(c2)
c2 = tf.keras.layers.Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c2)
p2 = tf.keras.layers.MaxPooling2D((2, 2))(c2)

c3 = tf.keras.layers.Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p2)
c3 = tf.keras.layers.Dropout(0.2)(c3)
c3 = tf.keras.layers.Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c3)
p3 = tf.keras.layers.MaxPooling2D((2, 2))(c3)

c4 = tf.keras.layers.Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p3)
c4 = tf.keras.layers.Dropout(0.2)(c4)
c4 = tf.keras.layers.Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c4)
p4 = tf.keras.layers.MaxPooling2D(pool_size=(2, 2))(c4)

c5 = tf.keras.layers.Conv2D(256, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p4)
c5 = tf.keras.layers.Dropout(0.3)(c5)
c5 = tf.keras.layers.Conv2D(256, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c5)

# Expansive path
u6 = tf.keras.layers.Conv2DTranspose(128, (2, 2), strides=(2, 2), padding='same')(c5)
u6 = tf.keras.layers.concatenate([u6, c4])
c6 = tf.keras.layers.Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u6)
c6 = tf.keras.layers.Dropout(0.2)(c6)
c6 = tf.keras.layers.Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c6)

u7 = tf.keras.layers.Conv2DTranspose(64, (2, 2), strides=(2, 2), padding='same')(c6)
u7 = tf.keras.layers.concatenate([u7, c3])
c7 = tf.keras.layers.Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u7)
c7 = tf.keras.layers.Dropout(0.2)(c7)
c7 = tf.keras.layers.Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c7)

u8 = tf.keras.layers.Conv2DTranspose(32, (2, 2), strides=(2, 2), padding='same')(c7)
u8 = tf.keras.layers.concatenate([u8, c2])
c8 = tf.keras.layers.Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u8)
c8 = tf.keras.layers.Dropout(0.1)(c8)
c8 = tf.keras.layers.Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c8)

u9 = tf.keras.layers.Conv2DTranspose(16, (2, 2), strides=(2, 2), padding='same')(c8)
u9 = tf.keras.layers.concatenate([u9, c1], axis=3)
c9 = tf.keras.layers.Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u9)
c9 = tf.keras.layers.Dropout(0.1)(c9)
c9 = tf.keras.layers.Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c9)

outputs = tf.keras.layers.Conv2D(1, (1, 1), activation='sigmoid')(c9)
```

Alap U-net eredmények

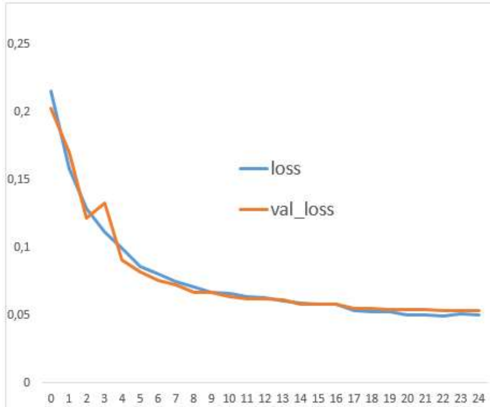
Optimalizálás:

- **batch size: 16**
- **learning rate: 0,00001**
- **optimizer: Adam**
- **loss function: binary cross entropy**






```
Precision score= 0.73
Recall Score= 0.756
Accuracy Score= 0.764
Dice Score= 0.743
Intersection Over Union= 0.591
F1 score= 0.743
```

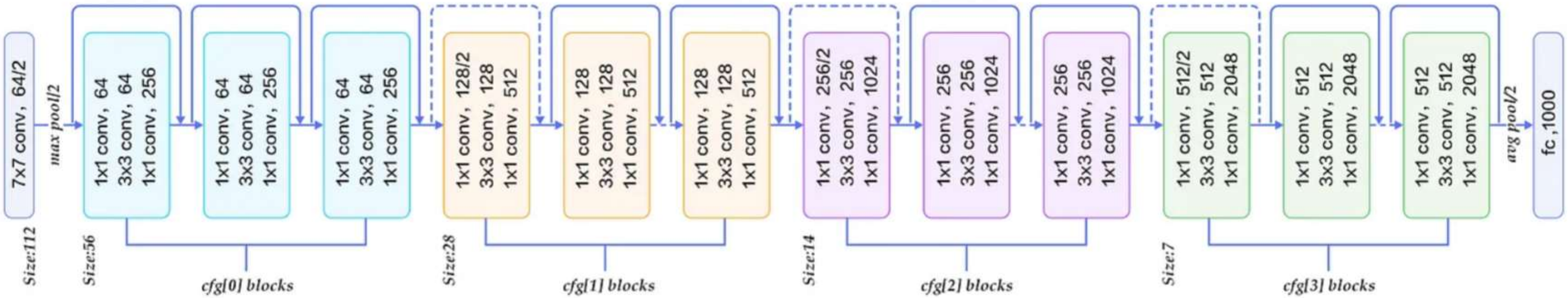


Pannon Egyetem
University of Pannonia

ResNet

A modell további javítása ResNet50 backbone-al.

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	[3×3, 64; 3×3, 64] ×2	[3×3, 64; 3×3, 64] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3
conv3_x	28×28	[3×3, 128; 3×3, 128] ×2	[3×3, 128; 3×3, 128] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×8
conv4_x	14×14	[3×3, 256; 3×3, 256] ×2	[3×3, 256; 3×3, 256] ×6	[1×1, 256; 3×3, 256; 1×1, 1024] ×6	[1×1, 256; 3×3, 256; 1×1, 1024] ×23	[1×1, 256; 3×3, 256; 1×1, 1024] ×36
conv5_x	7×7	[3×3, 512; 3×3, 512] ×2	[3×3, 512; 3×3, 512] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

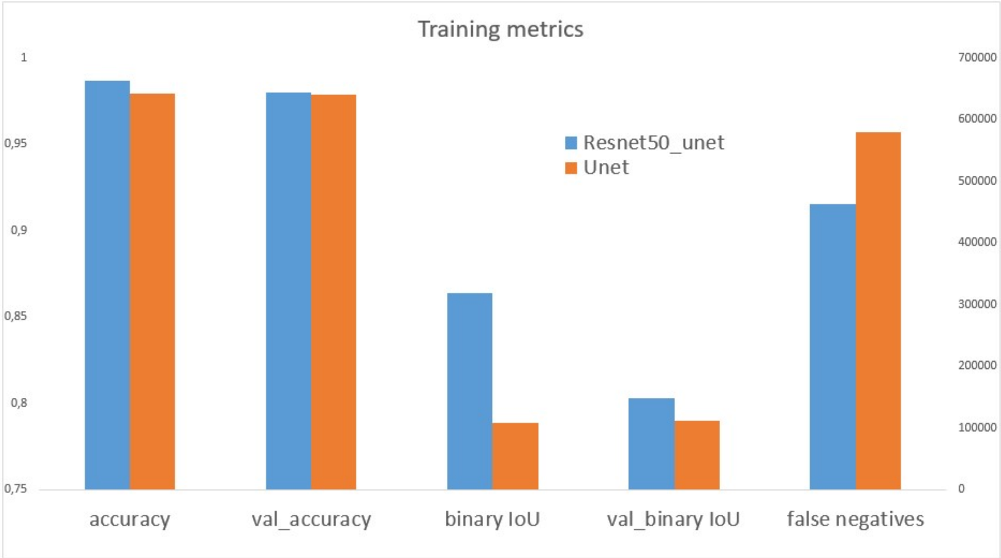


U-net + ResNet eredmények

```
def resnet50_encoder(input_shape):
    base_model = applications.ResNet50(include_top=False, weights='imagenet', input_shape=input_shape)
    layer_names = [
        'conv1_relu',
        'conv2_block3_out',
        'conv3_block4_out',
        'conv4_block6_out',
        'conv5_block3_out'
    ]
    layers_output = [base_model.get_layer(name).output for name in layer_names]
    return models.Model(inputs=base_model.input, outputs=layers_output)
```

```
Total params: 108,410,437 (413.55 MB)
Trainable params: 36,119,105 (137.78 MB)
Non-trainable params: 53,120 (207.50 KB)
Optimizer params: 72,238,212 (275.57 MB)
```

```
Precision score= 0.813
Recall Score= 0.794
Accuracy Score= 0.984
Dice Score= 0.803
Intersection Over Union= 0.675
F1 score= 0.803
```



Pannon Egyetem
University of Pannonia

Augmentálás

A train képek 15%-a augmentálva, majd ezzel növelve az eredeti képmennyiség.

Módszerek:

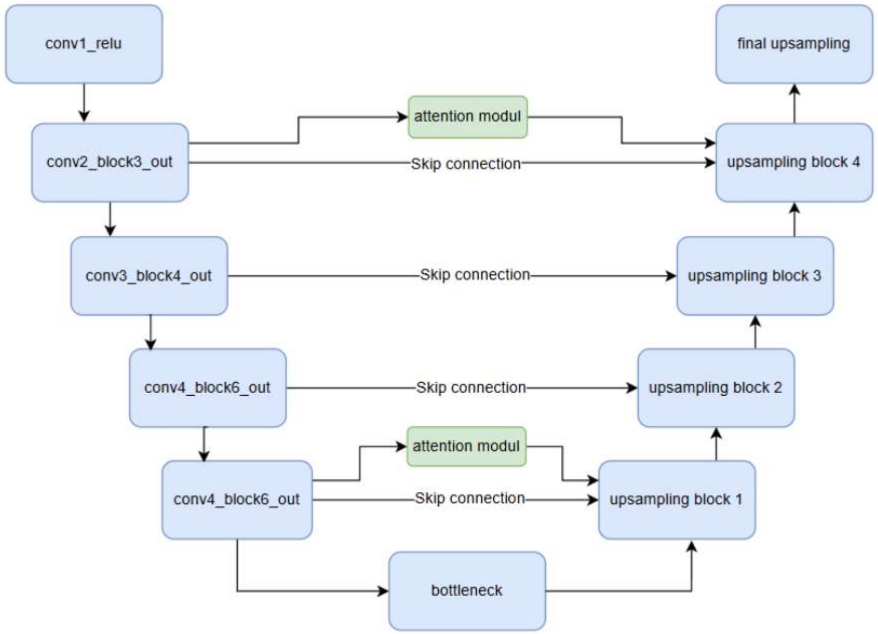
- **tf.image.flip_left_right – tükrözés**
- **tf.image.random_contrast(image, 0.2, 0.6)**
- **tf.image.random_brightness(image, max_delta=0.4)**

Attention modell

Térbeli fókusz, a jellemzők további kiemelésére szolgál

```
def spatial_attention_module(input_tensor):

    avg_pool = tf.keras.layers.GlobalAveragePooling2D(keepdims=True)(input_tensor)
    max_pool = tf.keras.layers.GlobalMaxPooling2D(keepdims=True)(input_tensor)

    concat = tf.keras.layers.Concatenate(axis=-1)([avg_pool, max_pool])
    attention_map = tf.keras.layers.Conv2D(1, (7, 7), padding='same', activation='sigmoid')(concat)

    return input_tensor * attention_map
```

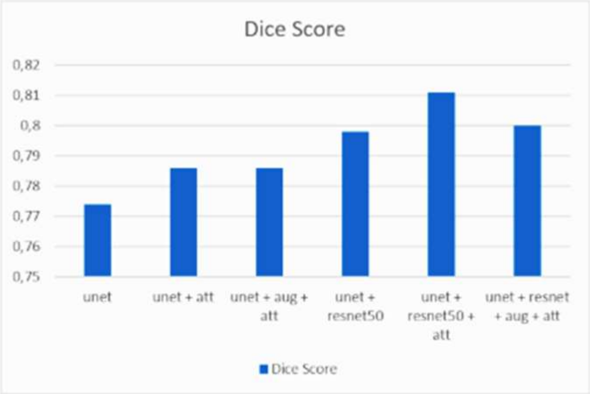
```
x = layers.Conv2DTranspose(512, (2, 2), strides=(2, 2), padding='same')(x)
skip1 = spatial_attention_module(encoder_outputs[-2])
x = layers.concatenate([x, skip1])
x = layers.Conv2D(512, (3, 3), activation='relu', padding='same')(x)
x = layers.Dropout(dropout_rate)(x)
x = layers.Conv2D(512, (3, 3), activation='relu', padding='same')(x)
```

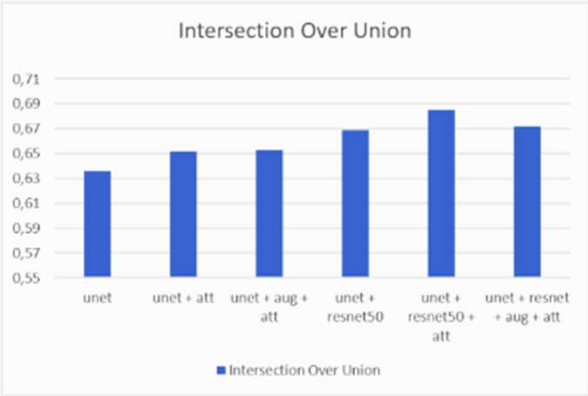


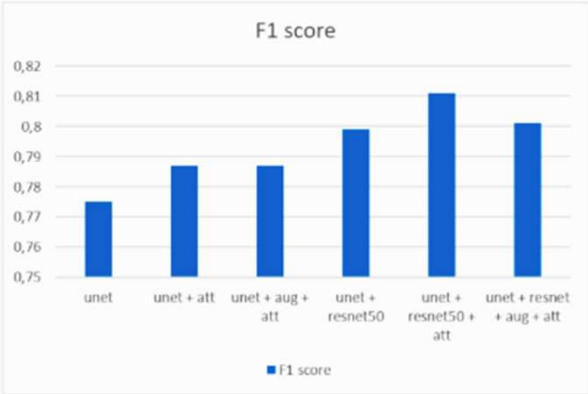
Pannon Egyetem
University of Pannonia

Eredmények

500db teszt kép eredményének az átlaga





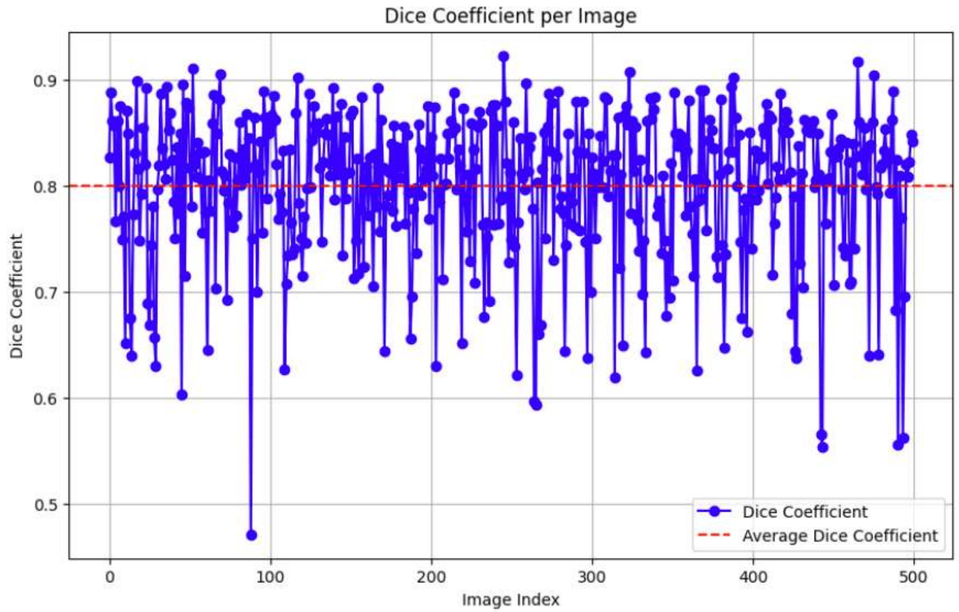


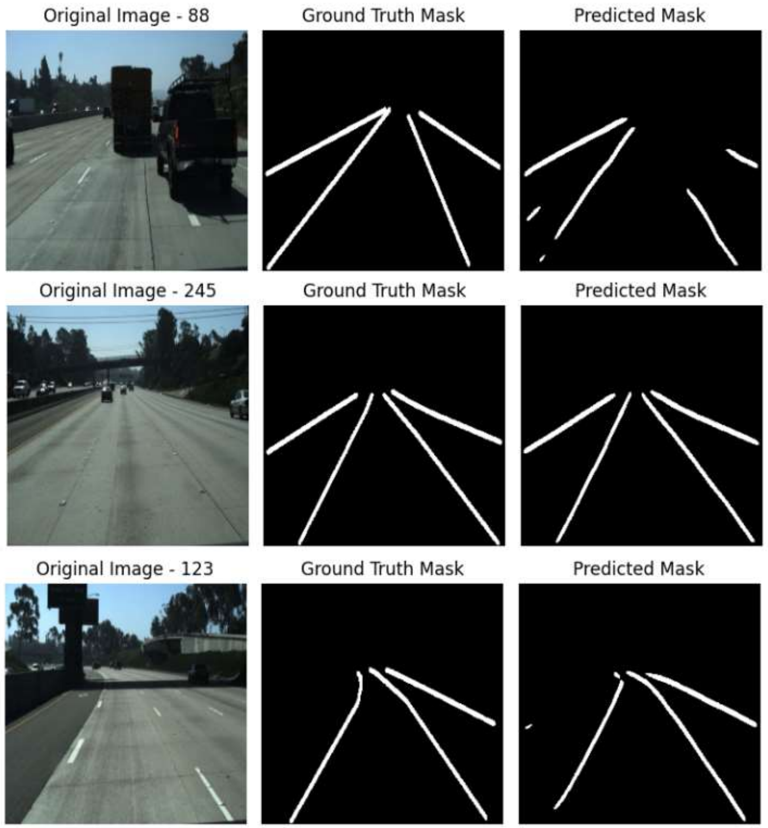
Model	Precision score	Recall Score	Accuracy Score	Dice Score	Intersection Over Union	F1 score
unet	0,793	0,758	0,981	0,774	0,636	0,775
unet + attention	0,802	0,772	0,982	0,786	0,652	0,787
unet + augmentation + attention	0,792	0,782	0,982	0,786	0,653	0,787
unet + resnet50	**0,828**	0,772	0,983	0,798	0,669	0,799
unet + resnet50 + attention	0,819	**0,804**	**0,984**	**0,811**	**0,685**	**0,811**
unet + resnet50 + augmentation + attention	0,830	0,774	0,983	0,800	0,672	0,801

Eredmények

500db teszt kép eredményének a szórása (Dice score):

- **minimum (kép 88: 0,471)**
- **maximum (kép 245: 0,923)**
- **átlag (kép 123: 0,811)**





Pannon Egyetem
University of Pannonia

Járműdetektálás

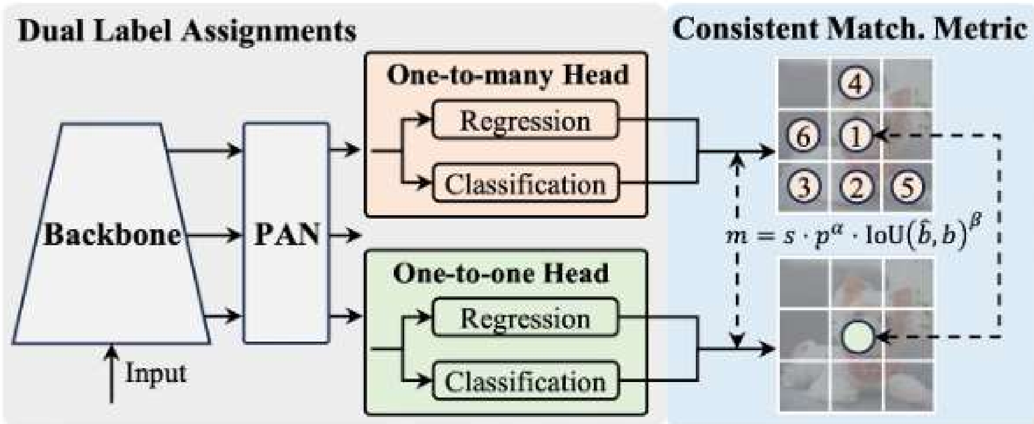
YOLO modell alkalmazása: (COCO dataset; 80db osztály)

Backbone: CSPNet

One-to-many head: sok predikció / objektum tanításkor

One-to-one head: 1 predikció / objektum futtatáskor

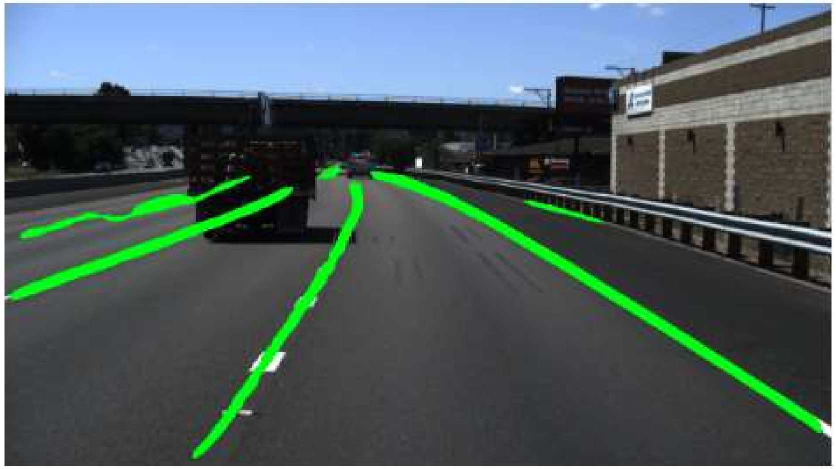
```
from ultralytics import YOLO

# Load a pre-trained YOLOv10n model
model = YOLO("yolov10n.pt")

# Perform object detection on an image
results = model("image.jpg")

# Display the results
results[0].show()
```

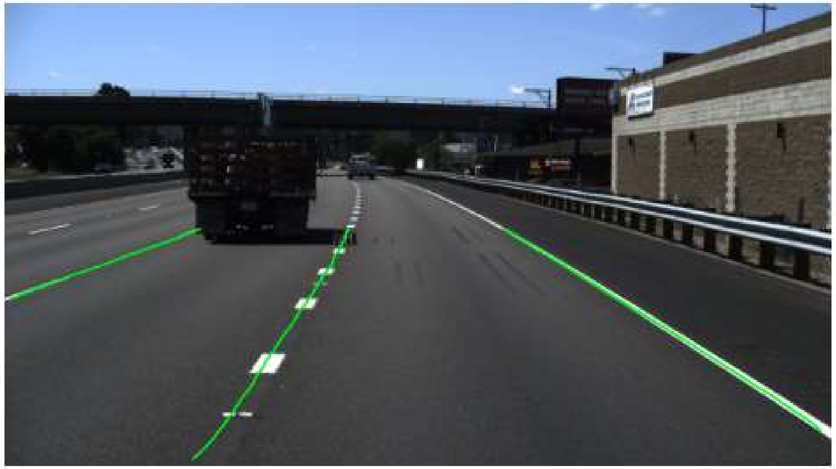


Sávdetektálás

A sáv kimeneti kép morfológiai javítása:

- **Blur**
- **Predikció**
- **Normalizálás - threshold**
- **Skeletonize**
- **Region Of Interest**
- **Countour extraction -> contour length**

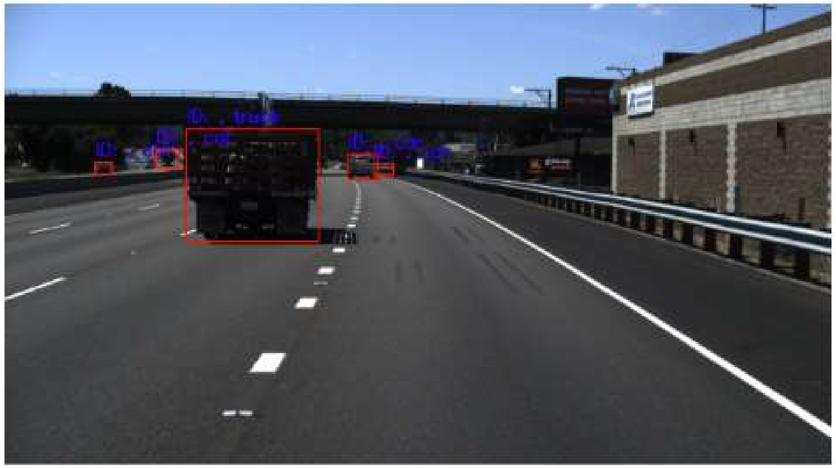


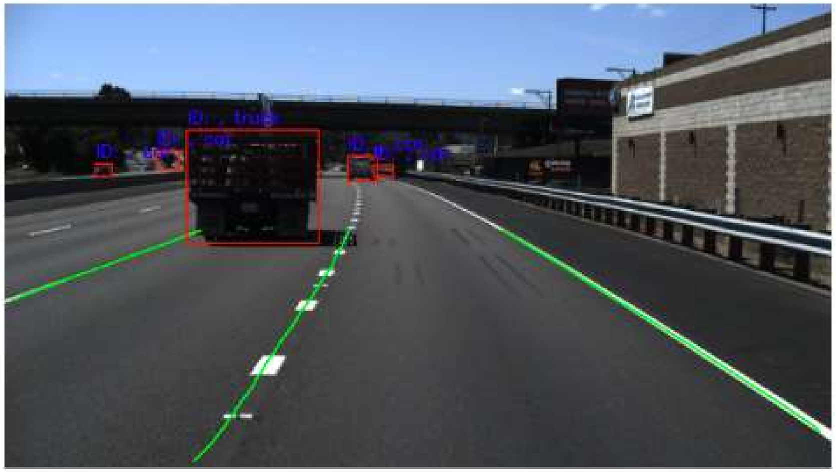


Járműdetektálás, a kettő kombinálása

Yolo predikcióból kiolvasni:

- **ID a követéshez**
- **Box koordináták**
- **Osztály feliratok**





Fejlesztés: saját sáv

1. Bounding box azonosítása (piros téglalap)
2. X szerinti középső koordináta keresése (sárga pont)
3. A kép középével való összehasonlítása (fehér pont), hogy bal vagy jobb oldali-e
4. A bal és jobb oldaliak közül a középponthoz legközelebbi kiválasztása
5. Azonosított kontúrok kirajzolása a képre